

Universidad de Concepción

Facultad de Ingeniería
Departamento de Ingeniería Informática y Ciencias de la Computación

Estructuras de Datos y Algoritmos Avanzados (2026-2)

Boletín 04: Range Queries

Autor:
Bryan Eliseo Aguirre Fuentes

Matrícula:
2024443402

Profesor:
Dr. José Fuentes

Ayudante:
Leonardo Lovera

Concepción, Chile
6 de noviembre de 2026

Índice

1. Resumen **1**
1.1. Propósito y alcance del estudio . . . 1
1.2. Estructuras y algoritmos estudiados . . . 1

2. Análisis teórico **1**
2.1. Sparse Table . . . 2
2.2. Segment Tree . . . 3
2.3. Comparación de complejidades . . . 3

3. Implementación **3**
3.1. Sparse Table . . . 4
3.2. Segment Tree . . . 4

4. Metodología experimental **4**
4.1. Características de la máquina . . . 4
4.2. Datos de entrada . . . 4
4.3. Experimentos e hipótesis . . . 5
4.3.1. Construcción inicial en el escenario estático . . . 6
4.3.2. Consultas en el escenario estático . . . 6
4.3.3. Mantenimiento en el escenario dinámico . . . 6
4.3.4. Costo dinámico de mantenimiento y consultas . . . 6
4.3.5. Análisis del espacio utilizado . . . 6

5. Resultados experimentales **7**
5.1. Escenario estático: construcción y consultas . . . 7
5.2. Escenario dinámico: mantenimiento y costo por ciclo . . . 8
5.3. Resumen numérico de tiempos y memoria . . . 9

6. ¿Qué opción utilizar? **10**

7. Generalización a otras operaciones binarias **11**

Referencias **13**

1. Resumen

1.1. Propósito y alcance del estudio

Este estudio compara dos estructuras de datos para resolver el problema de *Range Minimum Query* (RMQ): **Sparse Table** y **Segment Tree**. Dado un arreglo de enteros A, una consulta sobre el rango inclusivo $A[L..R]$ busca obtener el **índice del elemento mínimo**, no solamente su valor. Se consideran un **escenario estático**, en el que el arreglo permanece fijo, y un **escenario dinámico**, en el que sus elementos reciben modificaciones puntuales.

El diseño experimental contempla cuatro comparaciones:

- **Construcción en el escenario estático:** medir el tiempo necesario para construir ambas estructuras a partir del mismo arreglo, considerando diferentes cantidades de elementos.
- **Consultas en el escenario estático:** medir el tiempo de respuesta sobre las estructuras ya construidas, utilizando rangos aleatorios de distintas longitudes y arreglos de diferentes tamaños.
- **Mantenimiento en el escenario dinámico:** comparar el tiempo de una actualización puntual en Segment Tree con el tiempo de reconstruir Sparse Table después del mismo cambio. Esta comparación representa el costo de mantener respuestas vigentes, no dos operaciones de actualización equivalentes.
- **Costo del escenario dinámico:** comparar la suma del tiempo de mantenimiento y las consultas del mismo ciclo, incluyendo una reconstrucción de Sparse Table o una actualización de Segment Tree.

En cada comparación se utilizaron las mismas entradas y consultas para ambas estructuras. Además, se realizó un **análisis del espacio utilizado**, considerando las asignaciones de memoria de los constructores y las reservas temporales relevantes durante la construcción o reconstrucción. Los resultados se interpretan junto con las complejidades teóricas para discutir las ventajas y limitaciones de cada alternativa. El estudio también aborda su adaptación a otras operaciones binarias y las propiedades algebraicas que permiten conservar sus garantías.

1.2. Estructuras y algoritmos estudiados

- **Sparse Table:** preprocesa el arreglo almacenando resultados para bloques de longitud potencia de dos. Para RMQ permite responder consultas en tiempo constante mediante la combinación de dos bloques que pueden solaparse. La implementación estudiada no ofrece actualizaciones, por lo que en el escenario dinámico se reconstruyó después de cada modificación.
- **Segment Tree:** organiza los mínimos de intervalos en una estructura jerárquica. Responde consultas y permite actualizar un elemento en tiempo logarítmico, recalculando los nodos afectados sin reconstruir toda la estructura.

Las implementaciones incorporadas proceden del repositorio oficial del curso, con autoría original de LELE [1, 2].

2. Análisis teórico

Sea A un arreglo no vacío de n enteros. Una consulta RMQ sobre $A[L..R]$, con $0 \leq L \leq R < n$, devuelve el índice de su elemento mínimo. Denotamos por $k = R - L + 1$ la longitud del rango y resolvemos los empates con el **menor índice**. Para ello, cada candidato se representa mediante el par $(A[i], i)$: el mínimo lexicográfico compara primero el valor y luego el índice. Las implementaciones

reciben intervalos semiabiertos $[L, R+1)$; convertir los extremos y extraer el índice de la respuesta cuesta $\Theta(1)$.

Usamos $O(\cdot)$ para cotas superiores y $\Theta(\cdot)$ para cotas ajustadas. Suponemos un modelo de palabras de máquina: valores, índices y pares ocupan espacio constante, y acceder, copiar y comparar un par cuesta $\Theta(1)$. El espacio de la estructura incluye sus copias internas, pero excluye el arreglo externo de entrada; devolver un índice ocupa $\Theta(1)$. Las expresiones con $\log n$, en base dos, se presentan para $n \geq 2$; con un solo elemento las operaciones RMQ son constantes. El análisis se deriva del código del curso y de las descripciones de ambas estructuras [1, 2, 3, 4].

2.1. Sparse Table

Sparse Table almacena respuestas para bloques de longitud potencia de dos. Si $S_j[i]$ es el mínimo lexicográfico del bloque que comienza en i y tiene 2^j elementos, su construcción utiliza

$$S_0[i] = (A[i], i), \qquad S_j[i] = \min_{\text{lex}}\big(S_{j-1}[i], S_{j-1}[i + 2^{j-1}]\big).$$

La implementación reserva $\ell = \lfloor \log_2 n \rfloor + 1$ filas completas de n elementos, aunque algunas posiciones no correspondan a bloques válidos [1]. Para una consulta, toma $b = \lfloor \log_2 k \rfloor$ y combina

$$\min_{\text{lex}}\big(S_b[L], S_b[R - 2^b + 1]\big).$$

Los dos bloques cubren el rango y pueden solaparse. La duplicación de su intersección no cambia el mínimo, por la idempotencia de esta operación [3].

- **Preparación (construcción):** $\Theta(n \log n)$, tanto en mejor como en peor caso. Inicializar las filas ya requiere $\Theta(n\ell)$ trabajo. Además, si $m = \lfloor \log_2 n \rfloor$, se realizan $\sum_{j=1}^{m}(n - 2^j + 1) = m(n+1) - 2^{m+1} + 2$ combinaciones; cada una cuesta tiempo constante.
- **Mejor caso de consulta RMQ:** $\Theta(1)$.
- **Peor caso de consulta RMQ:** $\Theta(1)$. El método `query_idempotent` realiza dos accesos y una combinación. Calcula el nivel con `std::bit_width`, que permite obtener $\lfloor \log_2 k \rfloor$ sobre una palabra de ancho fijo sin recorrer el rango [3, 6].
- **Consulta general:** el método distinto `query` descompone k en potencias de dos y combina bloques disjuntos. Procesa tantos bloques como bits uno tiene k: su costo es $\Theta(\text{popcount}(k))$, con mejor caso $\Theta(1)$ y cota $O(\log(k+1))$. No es la variante utilizada para medir RMQ.
- **Mantenimiento tras un cambio:** la implementación no ofrece `update`. En el escenario dinámico se reconstruye la tabla, con costo $\Theta(n \log n)$ por modificación, independientemente de su posición. Es una estrategia de mantenimiento del estudio, no una operación de actualización nativa ni una imposibilidad de diseñar otras variantes.
- **Memoria:** $\Theta(n \log n)$ persistente y $\Theta(1)$ auxiliar por consulta. Para $s = \text{sizeof}(T)$, las filas contienen $n\ell$ objetos de tipo T, cuyo tamaño lógico de datos es $n\ell s$ bytes, además de los metadatos de los vectores y del combinador. La expresión que crea las filas utiliza también un vector temporal de n elementos: $\Theta(n)$ espacio temporal, sin cambiar el orden $\Theta(n \log n)$ del pico total de construcción.

El tamaño lógico no debe confundirse con los bytes efectivamente reservados: estos dependen de las capacidades de los vectores y de la biblioteca usada. Si durante una reconstrucción coexisten la tabla anterior y la nueva, el pico puede aumentar, pero conserva el mismo orden asintótico.

2.2. Segment Tree

Segment Tree mantiene mínimos de grupos de elementos en una jerarquía binaria [4]. La versión del curso es iterativa: reserva un vector de $2n$ posiciones, coloca las hojas en $[n, 2n)$ y calcula los nodos internos combinando sus dos hijos. No redondea n a una potencia de dos [2].

- **Preparación (construcción):** $\Theta(n)$, tanto en mejor como en peor caso. Inicializa $2n$ posiciones, copia n hojas y calcula exactamente $n-1$ mínimos de hijos. No realiza n actualizaciones independientes, lo que costaría $O(n \log n)$.
- **Mejor caso de consulta:** $\Theta(1)$, por ejemplo al consultar un único elemento. En ese caso el intervalo se consume en una iteración del recorrido ascendente.
- **Peor caso de consulta:** $\Theta(\log n)$. Los dos extremos ascienden dividiéndose por dos y se incorporan como máximo dos nodos por nivel. Existen rangos que requieren recorrer $\Theta(\log n)$ niveles. Más precisamente, para longitud k el bucle queda acotado por $O(\log(k+1))$, porque la cantidad de posiciones pendientes se reduce al menos a la mitad en cada paso. Por ello, mantener k fijo no obliga a observar crecimiento logarítmico con n, aunque sí puedan variar los accesos a caché.
- **Actualización puntual:** $\Theta(\log n)$ en mejor y peor caso para $n \geq 2$. Después de escribir una hoja, el método `update` recalcula todos sus ancestros hasta la raíz. No termina anticipadamente si el nuevo valor coincide con el anterior o si un mínimo intermedio no cambia.
- **Memoria:** $\Theta(n)$ persistente y $\Theta(1)$ auxiliar por consulta y actualización. Los $2n$ objetos del vector representan $2ns$ bytes lógicos de datos, más los metadatos y cualquier capacidad adicional. La construcción usa $\Theta(1)$ espacio temporal fuera de ese almacenamiento, y el pico total es $\Theta(n)$. Las actualizaciones no crean nuevos nodos.

Estas cotas se refieren al código iterativo incorporado, no a otra implementación recursiva que reserve $4n$ posiciones. En ambas estructuras, el contenido de los valores no modifica las reservas principales ni la cantidad de pasos de construcción; el costo de consulta del árbol depende del rango, no de encontrar antes un valor especialmente pequeño.

2.3. Comparación de complejidades

La tabla resume RMQ con pares de tamaño fijo. La consulta excluye la construcción y el mantenimiento; en Sparse Table corresponde a `query_idempotent`. La columna de mantenimiento compara reconstruir Sparse Table con actualizar Segment Tree, no dos constructores.

Estructura	Construcción	Mejor consulta	Peor consulta	Mantenimiento	Memoria persistente
Sparse Table	$\Theta(n \log n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Segment Tree	$\Theta(n)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(n)$

3. Implementación

Las dos implementaciones son clases genéricas de C++20 obtenidas del repositorio del curso y conservadas en `include/range_queries/`. Reciben un vector de elementos de tipo `T` y consultan intervalos semiabiertos $[i, j)$. Para obtener el índice del mínimo se utilizaron pares $(\text{valor}, \text{índice})$, cuyo orden lexicográfico resuelve empates por el menor índice. Se reemplazaron los comentarios y se añadió la cabecera `<bit>` a Sparse Table, sin modificar la lógica de los algoritmos.

3.1. Sparse Table

La clase `sparse_table` guarda una matriz de vectores `ST` y una función de combinación `f`. El constructor obtiene $m = \lfloor \log_2 n \rfloor$ mediante `_lg`, reserva $m+1$ filas de n elementos y copia la entrada en `ST[0]`. Dos ciclos construyen las filas siguientes: `p` representa la longitud de un bloque y `q` el doble; cada posición válida combina dos bloques consecutivos de la fila anterior.

Para RMQ se empleó `query_idempotent`, que calcula el nivel `b` a partir de la longitud del intervalo y combina los bloques de esa fila que comienzan en i y $j - 2^b$. La clase también incluye `query`, que recorre bloques disjuntos, pero no es la variante seleccionada para esta comparación. Como no dispone de `update`, en el escenario dinámico se reconstruyó desde el arreglo modificado. La implementación proviene del repositorio del curso [1].

3.2. Segment Tree

La clase `segment_tree` almacena la estructura en un vector `ST` de $2n$ elementos. El constructor copia la entrada en las hojas, desde `ST[n]` hasta `ST[2n - 1]`, y recorre los índices internos en orden descendente para guardar el mínimo de cada par de hijos mediante `_min`.

El método `query` desplaza los extremos del intervalo a las hojas e inicializa `q` con un elemento del rango. Un ciclo asciende por el árbol, incorpora los nodos de los bordes según la paridad de los extremos y conserva el menor candidato. El método `update` cambia una hoja y recalcula el mínimo de sus ancestros, combinando cada nodo con su hermano hasta alcanzar la raíz, sin reconstruir el vector. La implementación proviene del repositorio del curso [2].

4. Metodología experimental

4.1. Características de la máquina

Los experimentos se realizaron en una estación de trabajo con las siguientes características:

- **Procesador:** Intel Core i5-13420H (Raptor Lake), 8 núcleos y 12 hilos (4 P-Cores y 4 E-Cores), frecuencia base de 2.10 GHz y turbo de hasta 4.60 GHz. Cachés: L1d de 320 KiB, L1i de 384 KiB, L2 de 7 MiB y L3 compartida de 12 MiB.
- **Memoria principal:** 16 GiB DDR (15 GiB utilizables por el sistema operativo).
- **Almacenamiento:** SSD NVMe WD PC SN740 de 512 GB, interfaz PCIe Gen4 $\times 4$.
- **Entorno de compilación:** Arch Linux x86_64, kernel `7.2.14-arch1-1`, GCC 16.2.1 y las banderas `-std=c++20 -O3 -march=native -Wall -Wextra -Wpedantic -Iinclude`.
- **Afinidad de CPU:** se fijó el proceso de medición al núcleo lógico 2 mediante `taskset -c 2`. Esto reduce las migraciones entre núcleos, pero no elimina variaciones de frecuencia, temperatura ni la actividad concurrente del sistema.

4.2. Datos de entrada

Se generaron arreglos de **1.000, 4.000, 16.000, 64.000, 256.000 y 1.000.000 de enteros**, con valores uniformes en $[0, 10^9]$ y tres semillas fijas: **1729, 2718 y 31415**. Los auxiliares de `include/utils/rmq_inputs.hpp` adaptan las plantillas de los boletines 1 y 2, utilizando `mt19937_64` y muestreo por rechazo para evitar sesgo. Cada elemento es un par de enteros de 64 bits (valor, índice); el mínimo lexicográfico devuelve la posición más a la izquierda en caso de empate. Las semillas derivadas separan las secuencias de arreglos, consultas y modificaciones.

Se usaron longitudes $k \in \{1, 8, 63, 511, \lfloor n/2 \rfloor, n\}$. Para cada consulta se eligió L uniformemente en $[0, n-k]$ y se fijó $R = L + k - 1$, traduciendo $[L, R]$ a $[L, R+1)$ para invocar el código. El rango completo es un caso especial con una sola posición posible. En el escenario estático se prepararon **10.000 consultas por longitud**; en el dinámico, las posiciones se muestrearon uniformemente en $[0, n-1]$, con reemplazo, y el nuevo valor perteneció a $[0, 10^9]$ y fue distinto del vigente, conservando su índice. Ambas estructuras recibieron exactamente los mismos arreglos, cambios y consultas.

La generación y la conversión a pares quedaron fuera del cronometraje. Se registraron configuración, semillas, versiones y huellas SHA-256 de fuentes, ejecutables y entradas. La corrección se comprobó exhaustivamente en arreglos pequeños, incluidos casos crecientes, decrecientes y con empates; en la batería se contrastaron hasta ocho consultas por longitud y estado contra un barrido lineal independiente, calculado antes de medir.

4.3. Experimentos e hipótesis

Las hipótesis siguientes son las predicciones formuladas antes de la ejecución; su contraste con las mediciones se presenta en la sección de elección de estructura.

La ejecución se orquestó con scripts Bash basados en el boletín 3; Python se utilizó para registrar metadatos y generar tablas y figuras. Se realizaron **32 repeticiones por configuración y semilla**, después del calentamiento, siguiendo el esquema `uhr` de LELE con `std::chrono::steady_clock` y nanosegundos [9]. En cada repetición se midió la construcción inicial y se reutilizaron las instancias primero para consultas estáticas y después para el recorrido dinámico. Cada método recorrió su carga por separado, sin mantener viva la otra estructura; su orden se alternó entre repeticiones. Generación, validación, escritura, restauración y destrucción quedaron fuera del reloj; la reconstrucción de Sparse Table midió crear la nueva tabla, liberando la anterior después. Se consumieron las respuestas para evitar su eliminación por el compilador. El tiempo por consulta incluye el recorrido del lote y el cálculo de su checksum, sin descontar ese costo común.

Los lotes estáticos se calibraron fuera de muestra hasta **5 ms** o un máximo de **1.000.000 de consultas**, recorriendo cíclicamente los intervalos preparados y manteniendo el lote fijo entre repeticiones de cada método y configuración. Las 216 calibraciones alcanzaron el objetivo. El tiempo se dividió por el número de operaciones efectivamente ejecutadas.

Cada recorrido dinámico tuvo **10 ciclos de una actualización seguida de 1.000 consultas**, repartidas equilibradamente entre las seis longitudes: 10 cambios y 10.000 consultas medidas por estructura y repetición. Cada estado actualizado sirvió para todos los lotes, sin reconstruir por longitud ni repetir el cambio al medir consultas. No se forzó que los rangos incluyeran la posición modificada; esta se verificó con una consulta unitaria fuera del reloj. Las repeticiones reiniciaron el mismo estado y secuencia; no se añadieron consultas dinámicas para calibrar.

Se conservaron CSV crudos y resúmenes con **media, desviación estándar muestral, mínimo, Q1, mediana, Q3 y máximo**, usando la convención de cuartiles de `uhr`. Los diez ciclos se agregaron dentro de cada repetición, normalizando por operaciones, no como muestras independientes. Cada semilla conserva su resumen. Las figuras muestran **dos curvas**, una por estructura: cada punto es la media de las tres medias por semilla, con igual peso, y las barras representan la desviación estándar muestral entre esas tres medias, **no intervalos de confianza** ni la dispersión de 96 ejecuciones independientes. Las 32 repeticiones caracterizan la variabilidad dentro de cada semilla, pero no garantizan significancia estadística. Los CSV conservan nanosegundos y bytes; para facilitar la lectura se usan milisegundos en construcción, mantenimiento y ciclos, nanosegundos por consulta y MiB en las figuras de memoria.

4.3.1. Construcción inicial en el escenario estático

Se varió n, con los mismos pares de entrada y tres semillas para ambas estructuras. Se midió el tiempo por construcción, sin cargar ese costo a las consultas de las instancias reutilizadas. **Hipótesis.** Sparse Table tardará más al crecer n, por su preparación $\Theta(n \log n)$, frente a $\Theta(n)$ de Segment Tree. Ese costo adicional puede compensarse con suficientes consultas estáticas, pero no se presupone que ocurra para cualquier carga.

4.3.2. Consultas en el escenario estático

Se estudió n con k fijo y k con n fijo, sin cambios y con consultas idénticas. Los rangos medio y completo se separan porque crecen con n. Se midió el tiempo por consulta de `query_idempotent` frente a `query` del árbol, sin construcción. **Hipótesis.** Sparse Table favorecerá las consultas por su costo $\Theta(1)$, frente al peor caso $\Theta(\log n)$ del árbol. No se espera esa diferencia en todos los rangos: los unitarios son constantes en ambas estructuras y mantener k fijo no obliga al árbol a crecer con n.

4.3.3. Mantenimiento en el escenario dinámico

Se varió n, reproduciendo diez cambios y sus consultas, sin agrupar modificaciones antes de reconstruir. Se midió el tiempo por cambio: `update` del árbol frente a reconstrucción de la tabla, excluyendo consultas y construcción inicial. **Hipótesis.** Segment Tree tendrá menor tiempo de mantenimiento por su actualización $\Theta(\log n)$, frente a reconstruir Sparse Table en $\Theta(n \log n)$; la diferencia debería crecer con n. Se comparan estrategias de mantenimiento, no dos actualizaciones nativas.

4.3.4. Costo dinámico de mantenimiento y consultas

Se varió n con los mismos cambios y lotes de 1.000 consultas repartidas entre longitudes. El tiempo por ciclo suma mantenimiento y consultas del mismo método, semilla y repetición, contando una sola reconstrucción o `update`. Los diez ciclos se promedian dentro de cada repetición antes de resumir las 32 repeticiones por semilla. La suma utiliza intervalos ya cronometrados y excluye la construcción inicial; no es una medición de extremo a extremo, pues generación, validación, escritura y liberación permanecen fuera del reloj. Se conserva el detalle de consultas para auditar sus componentes, pero no se interpreta como el costo completo del escenario dinámico. **Hipótesis.** Con 1.000 consultas por cambio, el menor costo de mantenimiento de Segment Tree debería dominar para arreglos grandes. Sparse Table podría compensar su reconstrucción en arreglos pequeños; no se presupone un ganador para todo tamaño.

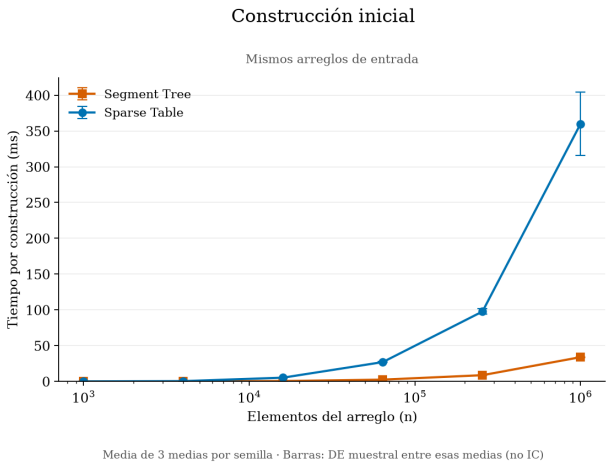
4.3.5. Análisis del espacio utilizado

En una pasada independiente se registraron bytes retenidos y picos de asignación, sin activar el seguimiento durante el cronometraje ni incluir la entrada externa o metadatos del asignador. Se distinguieron nuevas reservas y memoria previa al reconstruir; no son medidas de memoria residente total. **Hipótesis.** Sparse Table ocupará más espacio, por sus $n(\lfloor \log_2 n \rfloor + 1)$ pares frente a los $2n$ del árbol. Las actualizaciones reutilizan el vector, mientras que reconstruir exige nuevas reservas y puede elevar el pico al coexistir ambas versiones. Este análisis acompaña los cuatro experimentos, sin añadir otro de tiempo.

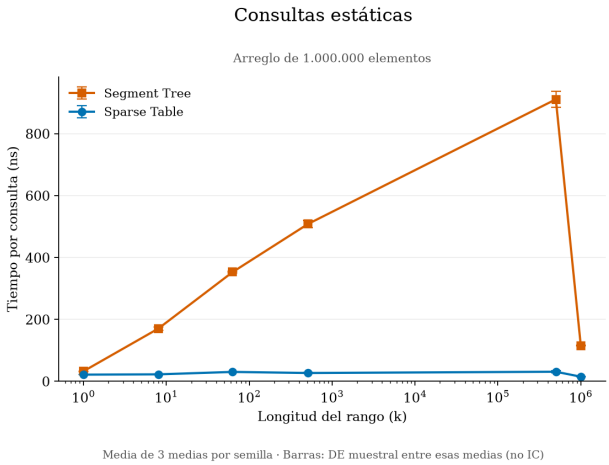
5. Resultados experimentales

La sesión oficial `official-20261106-185540-GmoOWTDo` completó las 32 repeticiones de cada configuración y semilla: 88.704 muestras temporales y 72 registros de memoria. Se seleccionan cuatro gráficos de los 18 disponibles; los restantes y los CSV se conservan en las carpetas de cada experimento. Cada curva corresponde a una estructura y cada punto promedia, con igual peso, las medias de las tres semillas. Las barras indican la desviación estándar muestral entre esas tres medias, no un intervalo de confianza. El eje horizontal es logarítmico y el vertical es lineal.

5.1. Escenario estático: construcción y consultas



(a) Experimento 1: construcción inicial.



(b) Experimento 2: consultas con $n = 10^6$.

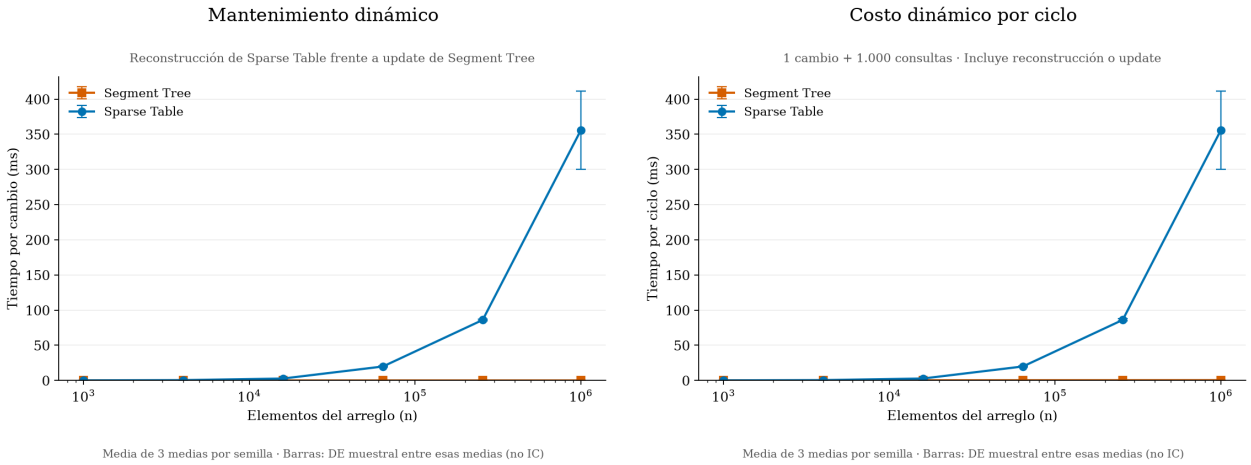
Figura 1: Escenario estático: costo de preparación y consultas sobre las estructuras ya construidas.

Experimento 1. Segment Tree presentó menor tiempo de construcción en todos los tamaños medidos. Para $n = 10^6$, su media fue 33,953 ms, frente a 359,880 ms de Sparse Table: una diferencia aproximada de 10,6 veces. La tendencia es consistente con las cotas $\Theta(n)$ y $\Theta(n \log n)$, respectivamente, aunque estos datos no demuestran por sí solos una complejidad asintótica.

Experimento 2. Sparse Table compensó parte de ese costo al consultar: para $n = 10^6$, sus medias estuvieron entre 14,765 y 30,814 ns, frente a 32,395–910,696 ns de Segment Tree. En el rango de longitud 511, por ejemplo, se observaron 26,714 y 507,993 ns. Este resultado se refiere a consultas con la estructura preparada, no al costo total de una aplicación estática. El rango completo fue más rápido que el medio en ambas estructuras; siempre consulta el mismo intervalo $[0, n)$, por lo que no debe interpretarse la curva como un crecimiento monótono con la longitud.

La preparación y el espacio limitan la ventaja de consultar más rápido: con un millón de elementos, Sparse Table retuvo 305,18 MiB, frente a 30,52 MiB de Segment Tree. Los picos durante la construcción fueron 320,44 MiB y 30,52 MiB, respectivamente. La tabla 4 resume el espacio sin añadir otra figura al informe.

5.2. Escenario dinámico: mantenimiento y costo por ciclo



(a) Experimento 3: reconstrucción frente a `update`. (b) Experimento 4: un cambio y 1.000 consultas.

Figura 2: Escenario dinámico: mantenimiento aislado y costo combinado de mantenimiento y consultas del mismo ciclo.

Experimento 3. Reconstruir Sparse Table fue más costoso que actualizar Segment Tree en todos los tamaños. Para $n = 10^6$, las medias fueron 355,673 ms por reconstrucción y 0,831 μs por `update`. Las unidades distintas de la tabla 1 evitan redondear a cero las actualizaciones del árbol. Su curva casi coincide con el eje horizontal en la figura por la diferencia de escalas, no porque el mantenimiento sea gratuito.

Experimento 4. Al incluir el cambio y las 1.000 consultas de cada ciclo, Sparse Table fue más rápida únicamente con 1.000 elementos: 0,105 ms frente a 0,156 ms. Segment Tree obtuvo menor tiempo desde 4.000 elementos entre los tamaños probados. Para $n = 10^6$, los costos fueron 355,736 y 0,387 ms, respectivamente, unas 919 veces a favor del árbol. El cambio de ganador se encuentra entre 1.000 y 4.000 elementos para esta carga; no se determinó su posición exacta ni se generaliza a otra proporción de modificaciones y consultas.

El costo por ciclo suma los intervalos de mantenimiento y consultas del mismo método, tamaño, semilla, repetición y ciclo. La reconstrucción se cuenta una sola vez, no por longitud de rango. Se excluyen la construcción inicial, generación, validación, escritura y liberación: es una suma de componentes medidos, no un cronómetro continuo de extremo a extremo. Los diez ciclos se promedian dentro de cada repetición antes de obtener la media por semilla; no constituyen diez repeticiones independientes.

En conjunto, las mediciones favorecen Sparse Table para consultar datos estáticos y Segment Tree para esta carga dinámica sobre arreglos grandes. La desviación entre semillas describe diferencias de entrada y condiciones de ejecución que el diseño no permite separar causalmente; no se presenta como prueba de significancia estadística.

5.3. Resumen numérico de tiempos y memoria

Se presentan cuatro tablas con todos los tamaños medidos. Construcción y mantenimiento comparten las filas de n, sin mezclar sus mediciones. Los tiempos se expresan como media ± desviación estándar muestral entre las tres medias por semilla, con igual peso. Cada media por semilla procede de 32 repeticiones: no es la desviación de 96 ejecuciones agrupadas ni un intervalo de confianza. La variación dentro de cada semilla se conserva en `summary.csv`.

Cuadro 1: Experimentos 1 y 3: construcción inicial y mantenimiento por cambio.

	Construcción inicial		Mantenimiento	
Elementos n	Sparse Table [ms]	Segment Tree [ms]	Sparse Table [ms]	Segment Tree [μs]
1.000	0,090 ± 0,003	0,025 ± 0,001	0,087 ± 0,003	0,340 ± 0,015
4.000	0,444 ± 0,042	0,095 ± 0,010	0,437 ± 0,047	0,384 ± 0,039
16.000	5,317 ± 0,356	0,550 ± 0,047	2,733 ± 0,221	0,446 ± 0,041
64.000	27,058 ± 1,060	2,551 ± 0,156	19,841 ± 0,862	0,596 ± 0,005
256.000	97,741 ± 3,915	8,775 ± 0,011	86,037 ± 1,489	0,716 ± 0,010
1.000.000	359,880 ± 44,177	33,953 ± 0,419	355,673 ± 55,479	0,831 ± 0,026

El mantenimiento compara reconstruir Sparse Table con `update` de Segment Tree. Solo esta última columna está en microsegundos.

Cuadro 2: Experimento 2: tiempo por consulta estática con $n = 10^6$.

Rango k	Sparse Table [ns]	Segment Tree [ns]
1	21,436 ± 0,199	32,395 ± 0,938
8	22,387 ± 0,752	170,172 ± 5,207
63	30,239 ± 1,315	352,761 ± 2,321
511	26,714 ± 1,720	507,993 ± 11,615
500.000	30,814 ± 3,387	910,696 ± 26,032
1.000.000	14,765 ± 0,336	114,839 ± 1,023

Cuadro 3: Experimento 4: tiempo por ciclo de un cambio y 1.000 consultas.

Elementos n	Sparse Table [ms]	Segment Tree [ms]
1.000	$0{,}105 \pm 0{,}004$	$0{,}156 \pm 0{,}003$
4.000	$0{,}457 \pm 0{,}049$	$0{,}182 \pm 0{,}018$
16.000	$2{,}772 \pm 0{,}222$	$0{,}218 \pm 0{,}015$
64.000	$19{,}889 \pm 0{,}864$	$0{,}284 \pm 0{,}005$
256.000	$86{,}090 \pm 1{,}488$	$0{,}332 \pm 0{,}007$
1.000.000	$355{,}736 \pm 55{,}485$	$0{,}387 \pm 0{,}003$

Cuadro 4: Memoria retenida y pico total durante el mantenimiento.

	Sparse Table		Segment Tree	
Elementos n	Retenida [MiB]	Pico total [MiB]	Retenida [MiB]	Pico total [MiB]
1.000	0,15	0,32	0,03	0,03
4.000	0,73	1,53	0,12	0,12
16.000	3,42	7,08	0,49	0,49
64.000	15,63	32,23	1,95	1,95
256.000	70,31	144,53	7,81	7,81
1.000.000	305,18	625,61	30,52	30,52

6. ¿Qué opción utilizar?

Construcción y consultas estáticas. Los resultados respaldan las dos primeras hipótesis: Segment Tree se construyó más rápido en todos los tamaños, mientras que Sparse Table presentó menores tiempos por consulta en las seis longitudes evaluadas con $n = 10^6$. Para ese tamaño, construir costó 33,953 ms y 359,880 ms, respectivamente (tabla 1); en rangos de longitud 511, consultar costó 507,993 ns y 26,714 ns (tabla 2). Esto es consistente con la preparación $\Theta(n)$ del árbol frente a $\Theta(n \log n)$ de la tabla, y con las consultas RMQ $\Theta(1)$ de esta última. No implica que toda consulta del árbol cueste $\Theta(\log n)$: los rangos unitarios son constantes en ambas estructuras. Tampoco demuestra que la tabla amortice su preparación con cualquier cantidad de consultas.

Mantenimiento y carga dinámica. La tercera hipótesis también queda respaldada: actualizar Segment Tree fue más barato que reconstruir Sparse Table en todos los tamaños. Con $n = 10^6$, los tiempos fueron 0,831 μs y 355,673 ms por cambio (tabla 1). La cuarta hipótesis se cumple para la carga estudiada: con un cambio y 1.000 consultas por ciclo, el árbol ganó desde 4.000 elementos; con un millón, necesitó 0,387 ms frente a 355,736 ms. Sin embargo, Sparse Table ganó con 1.000 elementos (0,105 ms frente a 0,156 ms), por lo que no existe un ganador universal (tabla 3). El cruce observado entre 1.000 y 4.000 elementos no determina un umbral exacto ni se extrapola a otras cargas.

Espacio utilizado. La quinta hipótesis queda respaldada por las reservas medidas: con un millón de elementos, Sparse Table retuvo 305,18 MiB frente a 30,52 MiB del árbol, unas diez

veces más. Además, su pico al reconstruir alcanzó 625,61 MiB por la coexistencia de versiones y reservas temporales; Segment Tree mantuvo 30,52 MiB al actualizar (tabla 4). Son asignaciones de las estructuras, no la memoria residente total del proceso.

Ventajas y desventajas. La principal ventaja de Sparse Table es responder RMQ en tiempo constante y con menores tiempos observados por consulta; sus desventajas son la preparación y el espacio $\Theta(n \log n)$, junto con la ausencia de actualización nativa en la implementación evaluada, que obliga a reconstruir en este estudio. Segment Tree ofrece construcción y espacio $\Theta(n)$, además de actualizaciones puntuales $\Theta(\log n)$ sin nuevas reservas. Su desventaja es un mayor costo de consulta, con peor caso $\Theta(\log n)$, que puede acumularse en cargas intensivas de lectura.

En resumen. Conviene Sparse Table para datos inmutables y memoria suficiente, cuando el ahorro en muchas consultas amortice su preparación. Conviene Segment Tree si hay cambios frecuentes, memoria limitada o pocas consultas que no compensen construir la tabla. La decisión debe considerar construcción, mantenimiento y consultas en conjunto, no solo la operación más rápida de cada estructura. Estas conclusiones describen las mediciones y son compatibles con las hipótesis teóricas; no constituyen una prueba de complejidad asintótica ni de significancia estadística.

7. Generalización a otras operaciones binarias

Ambas estructuras admiten las operaciones propuestas, aunque no siempre conservan las consultas constantes de RMQ. Esta generalización es teórica: los experimentos midieron únicamente mínimos.

Propiedades algebraicas. Sea $f : S \times S \to S$ una operación cerrada y **asociativa**, $f(f(a, b), c) = f(a, f(b, c))$. Esta propiedad permite reagrupar bloques consecutivos: (S, f) es un semigrupo. Con un **neutro** e, $f(e, x) = f(x, e) = x$, se obtiene un monoide, que facilita inicializar acumuladores y representar rangos vacíos [5]. Para rangos no vacíos basta iniciar con el primer bloque.

La **idempotencia**, $f(x, x) = x$, permite solapar bloques sin alterar el resultado. No se necesitan inversos ni conmutatividad si se preserva el orden. Sin asociatividad, la descomposición habitual puede dar respuestas incorrectas; reemplazar el mínimo por f no basta.

Operaciones propuestas. MCD y MCM denotan máximo común divisor y mínimo común múltiplo. Todas las operaciones son asociativas y conmutativas. Sparse Table combina bloques solapados solo para las idempotentes; en otro caso usa bloques disjuntos [3].

Operación	Idempotente	Neutro	Consulta Sparse Table
Mínimo	Sí	$+\infty$	$\Theta(1)$
Máximo	Sí	0	$\Theta(1)$
Suma	No	0	$O(\log n)$
Producto	No	1	$O(\log n)$
XOR	No	0	$O(\log n)$
AND	Sí	$2^w - 1$	$\Theta(1)$
OR	Sí	0	$\Theta(1)$
MCD	Sí	0	$\Theta(1)$
MCM	Sí	1	$\Theta(1)$

Se suponen enteros no negativos de w bits para AND y OR, $\gcd(0, 0) = 0$ y $\mathrm{mcm}(0, x) = 0$. MCM es idempotente: $\mathrm{mcm}(x, x) = x$. Para mínimo, el neutro puede ser el máximo representable en un dominio acotado, o añadirse $+\infty$.

Garantías de Sparse Table. Construcción, espacio y reconstrucción por cambio conservan $\Theta(n \log n)$. Las consultas idempotentes siguen siendo $\Theta(1)$. Para suma, producto y XOR, `query` combina bloques disjuntos en orden: procesa popcount(k) bloques, con cota $O(\log(k+1)) \subseteq O(\log n)$. No debe usarse `query_idempotent`: sumar los bloques $[2, 3]$ y $[3, 5]$ devuelve 13 en vez de 10, por repetir el 3.

Garantías y adaptación de Segment Tree. La asociatividad permite conservar construcción y espacio $\Theta(n)$, consultas $O(\log n)$ y actualizaciones $\Theta(\log n)$ [4, 5]. En el código actual, sustituir `_min` requiere corregir `query`: inicializa con una hoja que vuelve a incluir, repetición inocua para mínimo pero incorrecta para suma, producto y XOR. Se debe iniciar con el neutro o el primer bloque procesado. Para operaciones no conmutativas hacen falta acumuladores izquierdo y derecho ordenados; las actualizaciones deben combinar hijo izquierdo y luego derecho, no según cuál fue modificado. Son adaptaciones propuestas, no cambios a los algoritmos medidos.

Límites del modelo. Las cotas suponen agregados y combinaciones de costo constante. Suma, producto y MCM pueden desbordarse; la aritmética modular no da el resultado exacto. Dividir antes de multiplicar en MCM no garantiza representabilidad [8]. Con precisión arbitraria deben contabilizarse el tamaño, la copia y la combinación de resultados. Al analizar bits, MCD y MCM también incorporan el costo aritmético [7]; las garantías de RMQ no se trasladan incondicionalmente a enteros ilimitados.

Referencias

[1] LELE. *Generic Sparse Table* [Código fuente]. Repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/550e2199f82e74e38d253eba2e64f8c72af38296/rmq/sparse_table.cpp.

[2] LELE. *Generic Segment Tree for RMQs* [Código fuente]. Repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/550e2199f82e74e38d253eba2e64f8c72af38296/rmq/segment_tree.cpp.

[3] Colaboradores de CP-Algorithms. *Sparse Table*. https://cp-algorithms.com/data_structures/sparse-table.html.

[4] Colaboradores de CP-Algorithms. *Segment Tree*. https://cp-algorithms.com/data_structures/segment_tree.html.

[5] AtCoder Library. *Segtree*: documentación de árboles de segmentos sobre monoides. https://atcoder.github.io/ac-library/production/document_en/segtree.html.

[6] Cppreference. *std::bit_width*. https://en.cppreference.com/w/cpp/numeric/bit_width.html.

[7] Colaboradores de CP-Algorithms. *Euclidean algorithm for computing the greatest common divisor*. https://cp-algorithms.com/algebra/euclid-algorithm.html.

[8] Cppreference. *std::lcm*. https://en.cppreference.com/w/cpp/numeric/lcm.html.

[9] Lovera, L. (LELE). (s. f.). *uhr: generic time performance tester* [Código fuente]. Plantilla de medición de la ayudantía, disponible en `utils/uhr.cpp` del repositorio de estos boletines. Referencia del autor utilizada en los boletines anteriores: https://github.com/leonardlover.

Declaración de uso de IA generativa. Se utilizaron herramientas de inteligencia artificial generativa como apoyo para las implementaciones de los experimentos y para revisar la redacción técnica de este informe. Codex se utilizó como agente prioritario durante este proceso. El autor es responsable de las decisiones finales, la verificación de los resultados y el contenido presentado.